

DATE:- 7/05/2026

Program 1

Robot Traversal Using Means-End Analysis

Solve a robot traversal problem on a 4x4 grid using Means-End Analysis. Identify the difference between the current and goal state, then apply the operator (Up/Down/Left/Right) that reduces it.

AI-Lab-6th-Sem-NIT-Srinagar > AI_Lab_08 > Q1.PY > ...

```
1 # Program 1: Robot Traversal using Means-End Analysis
2 current = [0, 0]
3 goal = [3, 3]
4 step = 1
5 print(f"Initial: ({current[0]},{current[1]}) Goal: ({goal[0]},{goal[1]}) diff: {goal[0]-current[0]},{goal[1]-current[1]}")
6 while current != goal:
7
8     dx = goal[0] - current[0]
9     dy = goal[1] - current[1]
10
11     if dx > 0:
12         current[0] += 1
13         move = "MoveRight"
14
15     elif dx < 0:
16         current[0] -= 1
17         move = "MoveLeft"
18
19     elif dy > 0:
20         current[1] += 1
21         move = "MoveUp"
22
23     elif dy < 0:
24         current[1] -= 1
25         move = "MoveDown"
26
27     print(f"Step {step}: {move} -> ({current[0]},{current[1]}) diff: {goal[0]-current[0]},{goal[1]-current[1]}")
28
29     step += 1
30 print("GOAL Reached")
31 print(f"Total moves:", step - 1)
```

OUTPUT:-

```
● gauravmathur@Annus-Mac Artificial_Intelligence_Lab % python -u "/Users/gauravmathur/Desktop/Artificial_
Initial: (0,0) Goal: (3,3) diff: 3,3
Step 1: MoveRight -> (1,0) diff: 2,3
Step 2: MoveRight -> (2,0) diff: 1,3
Step 3: MoveRight -> (3,0) diff: 0,3
Step 4: MoveUp -> (3,1) diff: 0,2
Step 5: MoveUp -> (3,2) diff: 0,1
Step 6: MoveUp -> (3,3) diff: 0,0
GOAL Reached
Total moves: 6
● gauravmathur@Annus-Mac Artificial_Intelligence_Lab %
```

Block-World Problem Using Means-End Analysis

Use MEA to solve the classic Block-World problem. The robot must rearrange three blocks from initial state to goal stack using pickup, putdown, stack, and unstack operators.

AI-Lab-6th-Sem-NIT-Srinagar > AI_Lab_08 > Q2.PY > ...

```

1  # Program 2: Block World using Means-End Analysis
2  state = {
3      "A": "table",
4      "B": "table",
5      "C": "table"
6  }
7  print("Initial State:")
8  print("A on table")
9  print("B on table")
10 print("C on table")
11 steps = []
12 # Step 1
13 steps.append("pickup(B)")
14 # Step 2
15 state["B"] = "A"
16 steps.append("stack(B, A)")
17 # Step 3
18 steps.append("pickup(C)")
19 # Step 4
20 state["C"] = "B"
21 steps.append("stack(C, B)")
22 print("\nGoal State:")
23 print("A on table")
24 print("B on A")
25 print("C on B")
26 print("\nSteps:")
27 for i, s in enumerate(steps, start=1):
28     print(f"Step {i}: {s}")
29
30 print("\nGOAL Achieved")
31 print("Total moves:", len(steps))

```

OUTPUT:—

```

● gauravmathur@Annus-Mac Artificial_Intelligence_Lab % python -u "/Users/gauravmathur/Desktop/Ar
.PY"
Initial State:
A on table
B on table
C on table

Goal State:
A on table
B on A
C on B

Steps:
Step 1: pickup(B)
Step 2: stack(B, A)
Step 3: pickup(C)
Step 4: stack(C, B)

GOAL Achieved
Total moves: 4
● gauravmathur@Annus-Mac Artificial_Intelligence_Lab %

```

Program 3

Travelling Salesman Problem Using Brute Force

Solve the TSP for 4 cities by enumerating all unique tours that start and end at city A. Print every tour with its cost and report the minimum-cost Hamiltonian cycle.

AI-Lab-6th-Sem-NIT-Srinagar > AI_Lab_08 > Q3.PY > ...

```

1  # Program 3: TSP using Brute Force
2  from itertools import permutations
3  cities = ['A', 'B', 'C', 'D']
4  dist = {
5      ('A', 'B'): 1,
6      ('A', 'C'): 4,
7      ('A', 'D'): 4,
8      ('B', 'A'): 1,
9      ('B', 'C'): 5,
10     ('B', 'D'): 1,
11     ('C', 'A'): 4,
12     ('C', 'B'): 5,
13     ('C', 'D'): 9,
14     ('D', 'A'): 4,
15     ('D', 'B'): 1,
16     ('D', 'C'): 9
17 }
18 min_cost = float('inf')
19 best_path = None
20 print("Tours from A:\n")
21 for perm in permutations(cities[1:]):
22     path = ['A'] + list(perm) + ['A']
23     cost = 0
24     for i in range(len(path)-1):
25         cost += dist[(path[i], path[i+1])]
26     print(" -> ".join(path), ":", cost)
27     if cost < min_cost:
28         min_cost = cost
29         best_path = path
30 print("\nOptimal Tour:")
31 print(" -> ".join(best_path))
32 print("Optimal Cost:", min_cost)

```

```

● gauravmathur@Annus-Mac Artificial_Intelligence_Lab % python -u "/Users/gauravmathur/Desktop/./PY"

```

Tours from A:

```

A -> B -> C -> D -> A : 19
A -> B -> D -> C -> A : 15
A -> C -> B -> D -> A : 14
A -> C -> D -> B -> A : 15
A -> D -> B -> C -> A : 14
A -> D -> C -> B -> A : 19

```

Optimal Tour:

```

A -> C -> B -> D -> A

```

Optimal Cost: 14

```

● gauravmathur@Annus-Mac Artificial_Intelligence_Lab %

```

Program 4

TSP — Brute Force vs Nearest Neighbour

Run Brute Force and the Nearest-Neighbour heuristic on the same 4-city graph. Compare paths and costs; observe that the greedy NN can miss the optimal tour.

```
AI-Lab-6th-Sem-NIT-Srinagar > AI_Lab_08 > Q4.PY > ...
1  # Program 4: TSP Brute Force vs Nearest Neighbour
2  from itertools import permutations
3  cities = ['A', 'B', 'C', 'D']
4  dist = {
5      ('A', 'B'): 1,
6      ('A', 'C'): 4,
7      ('A', 'D'): 4,
8      ('B', 'A'): 1,
9      ('B', 'C'): 5,
10     ('B', 'D'): 1,
11     ('C', 'A'): 4,
12     ('C', 'B'): 5,
13     ('C', 'D'): 9,
14     ('D', 'A'): 4,
15     ('D', 'B'): 1,
16     ('D', 'C'): 9
17 }
18 min_cost = float('inf')
19 best_path = None
20 for perm in permutations(cities[1:]):
21     path = ['A'] + list(perm) + ['A']
22     cost = 0
23     for i in range(len(path)-1):
24         cost += dist[(path[i], path[i+1])]
25     if cost < min_cost:
26         min_cost = cost
27         best_path = path
28 print("Brute Force:")
29 print("Tour :", " -> ".join(best_path))
30 print("Cost :", min_cost)
31 current = 'A'
32 visited = ['A']
33 nn_path = ['A']
34 nn_cost = 0
35 while len(visited) < len(cities):
36
37     nearest = None
38     nearest_cost = float('inf')
39
40     for city in cities:
41         if city not in visited and dist[(current, city)] < nearest_cost:
42             nearest = city
43             nearest_cost = dist[(current, city)]
44
45     visited.append(nearest)
46     nn_path.append(nearest)
47     nn_cost += nearest_cost
48     current = nearest
49     nn_path.append('A')
50     nn_cost += dist[(current, 'A')]
51 print("\nNearest Neighbour:")
52 print("Tour :", " -> ".join(nn_path))
53 print("Cost :", nn_cost)
54 error = ((nn_cost - min_cost) / min_cost) * 100
55 print("\nNN Error vs Optimal = {:.1f}%".format(error))
```

- gauravmathur@Annus-Mac Artificial_Intelligence_Lab % python -u "/Users/gauravmathur/Desktop/Artificial
Brute Force:
Tour : A -> C -> B -> D -> A
Cost : 14
- Nearest Neighbour:
Tour : A -> B -> D -> C -> A
Cost : 15
- NN Error vs Optimal = 7.1%
- gauravmathur@Annus-Mac Artificial_Intelligence_Lab %